

CS633 Assignment Group Number 33

Shlok Jain (230493), Akash Verma (220097), Krish Jain (230572),
Aditya Gautam (220064), Chaitanya Nitawe (230315)

1 Code Description

This program implements a distributed data exchange and computation pipeline using MPI. The solution adheres to strict blocking communication requirements (`MPI_Send` and `MPI_Recv`) without reliance on system buffering. To ensure deadlock-free execution in a linear topology with blocking calls, the code employs a **Block Parity Scheduling** algorithm.

1.1 Overview and Data Structures

The simulation runs for T iterations.

- **State Arrays:** `data_received_D1` and `data_received_D2` store the current data for the respective neighbors.
- **Buffers:** dedicated buffers for sending, receiving work, and returning results are allocated to prevent memory aliasing.
- **Role Definition:** At initialization, each rank calculates boolean flags (e.g., `is_sender_D1`, `is_worker_D2`) based on its rank and the problem topology boundaries.

1.2 Time-Efficient Communication (Block Parity)

The core challenge is performing blocking sends/receives simultaneously across a linear array efficiently. A naive approach (e.g., all ranks send, then receive) may cause a deadlock, or it may cause non-parallel execution if buffering is disabled. We solve this using a spatial scheduling approach:

1. We conceptually divide the linear array of processes into blocks of size D (where D is D_1 or D_2). The Block ID is calculated as `rank / D`.
2. Communication occurs in two synchronized phases:
 - **Phase A:** Even-numbered blocks act as **Initiators** (Senders send, Workers receive).
 - **Phase B:** Odd-numbered blocks act as **Initiators**.

This logic guarantees that at any given step, a process executing `MPI_Send` is paired with a neighbor executing `MPI_Recv`, preventing circular wait conditions.

```
1 // --- Handle D1 Forward ---
2 block_id = rank / D1;
3
4 // Phase A: Even blocks send, Odd blocks receive
5 if (block_id % 2 == 0) {
6     if (is_sender_D1) MPI_Send(..., rank + D1, ...);
7 } else {
8     if (is_worker_D1) MPI_Recv(..., rank - D1, ...);
9 }
10
11 // Phase B: Odd blocks send, Even blocks receive
12 if (block_id % 2 == 1) {
```

```

13  if (is_sender_D1) MPI_Send(..., rank + D1, ...);
14  } else {
15      if (is_worker_D1) MPI_Recv(..., rank - D1, ...);
16  }

```

Listing 1: Block Parity Logic for Forward Communication (D1)

This pattern is repeated for:

1. D_1 Forward (Send Work)
2. D_2 Forward (Send Work)
3. D_1 Backward (Return Results)
4. D_2 Backward (Return Results)

1.3 Computation and State Update

Between the forward and backward communication phases, "Worker" ranks perform the required computations:

- **D1 computation:** Squares the input values (x^2).
- **D2 computation:** Computes the natural logarithm ($\ln x$), handling domain errors where $x \leq 0$.

Upon receiving the results, the "Sender" ranks update their state. The results from D_1 overwrite `data_received.D1`, and results from D_2 overwrite `data_received.D2`.

1.4 Final Aggregation

After T iterations, the global maximum is computed.

- Each rank computes its local maximums for D_1 and D_2 .
- **Rank 0** iterates through all possible ranks. If a rank is a valid sender, Rank 0 executes a blocking `MPI_Recv`.
- All valid senders execute a blocking `MPI_Send` to Rank 0.

Rank 0 then compares all received values to find the global maximums and prints the result alongside the total execution time (reduced via `MPI_Reduce`).

2 Code Compilation and Execution Instructions

The code is written in C and utilizes the MPI standard. To compile and run the program, execute the following commands in the terminal:

```

# 1. Compile the code (linking the math library)
mpicc -o src src.c -lm

# 2. Submit the job script
sbatch job.sh

```

Note: The job script (`job.sh`) is configured to execute the binary file named `src`.

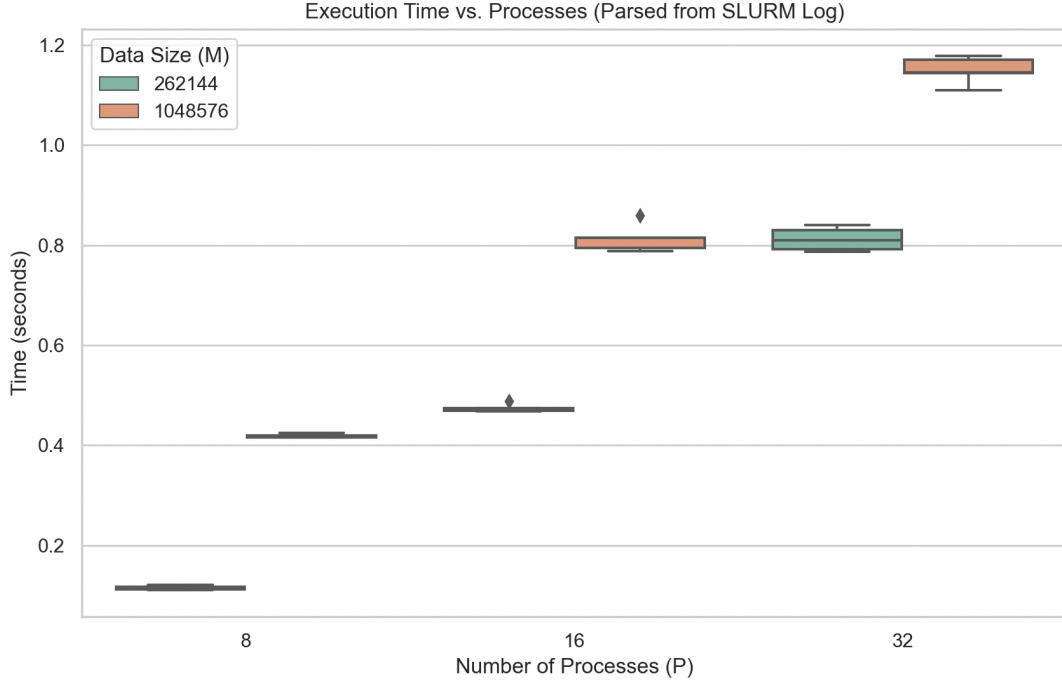


Table 1: Average Execution Time (seconds) for different Process Counts (P) and Data Sizes (M).

Processes (P)	Data Size (M)	Average Time (s)
8	262,144	0.114840
16	262,144	0.473862
32	262,144	0.811337
8	1,048,576	0.419108
16	1,048,576	0.814073
32	1,048,576	1.149736

3 Results

- **Impact of Data Size (M):** Execution time scales linearly with data size. Increasing M by a factor of 4 (262k to 1M) results in an approximate $3.8\times$ increase in time, confirming the computation complexity is $O(M)$.
- **Impact of Process Count (P):** Execution time increases as P grows. This is due to the cumulative synchronization overhead inherent in strict blocking communication and the sequential final data collection at Rank 0.
- **Topology Effects:**
 - At $P = 16$, the single node is fully saturated, leading to memory bandwidth contention.
 - At $P = 32$, the job spans two nodes (16 ranks/node). The significant jump in execution time is attributed to the latency of **inter-node network communication** compared to the faster shared-memory communication used at $P = 8$.
- **Stability:** The narrow boxplots indicate very low variance between the 5 execution runs, demonstrating that the program performance is deterministic and stable.

4 Conclusion

This assignment successfully implemented a distributed data exchange protocol using strict blocking MPI communication. We employed **Block Parity Scheduling**, which synchronizes processes across spatial blocks (sizes D_1 and D_2) to ensure that every sender is paired with a receiver.

The **contribution** to this assignment from all the members **was equal**.